Flash Sale Engine

Product Requirements Document & Sprint Plan

Author: Ujjwal Kumar Singh | Version 1.0

1. Project overview

The Flash Sale Engine is a backend system that lets an e-commerce platform run time-boxed, limited-stock sales events. Customers race to purchase a small number of heavily discounted units before either the timer or the stock runs out. The core engineering challenge is preventing overselling when thousands of requests arrive for the same product in the same second.

This document is organised the way a backend feature would be planned at a product company: epics group related capabilities, each epic breaks down into user stories with acceptance criteria, and stories map onto a phased delivery plan with sprint-level tickets. The phasing is deliberately aligned to a learning path — start with plain Spring Boot and MySQL, then introduce Redis for concurrency safety, then Kafka for async processing.

1.1 Actors

- Customer — registers, browses active/upcoming sales, and places orders during a flash sale window.
- Admin — creates products, schedules sales, and configures which products are part of a sale (price, stock, per-user limit).
- System (background worker) — processes queued orders asynchronously and reconciles inventory once a sale ends.

1.2 Glossary

- Sale — a scheduled, time-boxed flash sale event (has a start and end time).
- Product — a catalog item with a base (MRP) price, independent of any sale.
- Sale inventory — the join between a sale and a product: flash price, total stock, remaining stock, and max units per user for that sale.
- Order — a confirmed or pending purchase made by a user against a specific sale inventory record.

2. Epics and user stories

Each story below carries a ticket ID, priority, story point estimate, and the delivery phase it belongs to. Story points use a simple 1–8 scale (Fibonacci-ish) reflecting relative effort, not days.

Epic 1 — User management

US-01 — Register a new account

*As a **new customer**, I want **to create an account with my basic details**, so that **I can place orders during flash sales**.*

Acceptance criteria

- Given valid first name, last name, username, email, phone, and address fields, when I submit registration, then a user record is created with status ACTIVE.
- Given a username, email, or phone that already exists, when I submit registration, then the API returns a 409 Conflict with a clear field-level error.
- Given missing required fields, when I submit registration, then the API returns a 400 with field-level validation messages.

Ticket: FSE-101 **Priority:** High **Story points:** 5 **Phase:** Phase 1

US-02 — View and update my profile

*As a **registered customer**, I want **to view and update my contact and address details**, so that **my order deliveries go to the correct address**.*

Acceptance criteria

- Given I am authenticated, when I call GET /users/me, then I receive my profile excluding sensitive fields like password hash.
- Given I am authenticated, when I PATCH editable fields (phone, city, state, country, pin), then the record updates and updated_at changes.
- Given I attempt to change my username to one that already exists, when I submit the update, then the API returns a 409 Conflict.

Ticket: FSE-102 **Priority:** Medium **Story points:** 3 **Phase:** Phase 1

Epic 2 — Product catalog (admin)

US-03 — Manage product catalog

*As a **admin**, I want **to create, update, and deactivate products**, so that **products exist in the catalog before they can be added to a sale**.*

Acceptance criteria

- Given valid product details (name, description, image URL, category, base price, seller), when I create a product, then it is stored with status ACTIVE and a unique product_code is generated.
- Given an existing product, when I update its base price or description, then the change is reflected immediately for future sale configurations (existing sale_inventory flash prices are unaffected).
- Given a product I want to retire, when I set its status to INACTIVE, then it can no longer be added to new sales but remains visible in past order history.

Ticket: FSE-103 **Priority:** Medium **Story points:** 5 **Phase:** Phase 1

Epic 3 — Sale management (admin)

US-04 — Create and schedule a flash sale

*As a **admin**, I want **to create a sale with a title, description, start time, and end time**, so that **the sale becomes visible to customers at the right time and closes automatically**.*

Acceptance criteria

- Given a start time before an end time, when I create a sale, then it is stored with status SCHEDULED and a unique sale_code.
- Given the current time is between start_time and end_time, when the system evaluates sale status, then the sale is treated as ACTIVE.
- Given the current time is past end_time, when the system evaluates sale status, then the sale is treated as ENDED and no further orders are accepted against it.

Ticket: FSE-104 **Priority:** High **Story points:** 5 **Phase:** Phase 1

US-05 — Configure sale inventory for a product

*As a **admin**, I want **to add a product to a sale with a flash price, total stock, and a max-per-user limit**, so that **customers see the correct discounted price and stock cannot exceed what was allocated**.*

Acceptance criteria

- Given an existing sale and product, when I add a sale_inventory record with flash_price, total_stock, and max_per_user, then remaining_stock is initialised equal to total_stock.
- Given flash_price is greater than or equal to the product base_price, when I submit the configuration, then the API rejects it with a 400 (a flash price must be a real discount).
- Given a sale_inventory record already exists for a sale + product pair, when I try to add it again, then the API returns a 409 Conflict.

Ticket: FSE-105 **Priority:** High **Story points:** 5 **Phase:** Phase 1

Epic 4 — Sale discovery (customer)

US-06 — Browse active and upcoming sales

*As a **customer**, I want **to see a list of currently active and upcoming flash sales**, so that **I know when and where I can grab a deal**.*

Acceptance criteria

- Given sales with status ACTIVE or SCHEDULED, when I call GET /sales, then I receive them sorted by start_time with status, start_time, and end_time.
- Given a sale with status ENDED or CANCELLED, when I call GET /sales, then it is excluded from the default response (available via a history filter).

Ticket: FSE-106 **Priority:** Medium **Story points:** 3 **Phase:** Phase 1

US-07 — View products and live stock for a sale

*As a **customer**, I want **to see every product in a sale along with its flash price and remaining stock**, so that **I can decide what to buy and how urgently**.*

Acceptance criteria

- Given an active sale, when I call GET /sales/{id}/inventory, then I receive each product with name, image, base_price, flash_price, and remaining_stock.

- Given remaining_stock for an item reaches 0, when I fetch sale inventory, then that item is marked as sold out but still listed.

Ticket: FSE-107 **Priority:** Medium **Story points:** 3 **Phase:** Phase 1

Epic 5 — Order placement (core flow)

US-08 — Place an order during an active sale

*As a **customer**, I want **to place an order for a product in an active sale**, so that **I can purchase it at the flash price before stock runs out**.*

Acceptance criteria

- Given the sale is ACTIVE and remaining_stock > 0 for the requested sale_inventory, when I place an order, then an order record is created with status CONFIRMED, flash_price_paid snapshotted, and remaining_stock decremented by the order quantity.
- Given the sale is SCHEDULED or ENDED, when I place an order against it, then the API returns a 400 (sale not active).
- Given the requested quantity is greater than max_per_user minus my existing confirmed quantity for that sale_inventory, when I place an order, then the API returns a 409 with a clear message.

Ticket: FSE-108 **Priority:** High **Story points:** 8 **Phase:** Phase 1

US-09 — Prevent overselling

*As a **system**, I want **to guarantee that remaining_stock never goes below zero, even under concurrent requests**, so that **no two customers are sold the same last unit**.*

Acceptance criteria

- Given remaining_stock is 1 and two requests arrive at the same instant, when both attempt to decrement, then exactly one succeeds with status CONFIRMED and the other receives a 409 Sold Out.
- Given a load test of N concurrent requests against M available units, when the test completes, then exactly M orders are CONFIRMED and remaining_stock equals 0 (never negative).

Ticket: FSE-109 **Priority:** Critical **Story points:** 8 **Phase:** Phase 1 (DB-level), hardened in Phase 2

US-10 — View my order history

*As a **customer**, I want **to see all orders I have placed, across all sales**, so that **I can track what I bought and when**.*

Acceptance criteria

- Given I am authenticated, when I call GET /orders/me, then I receive my orders sorted by purchased_at descending, including sale title, product name, quantity, and amount paid.
- Given I have no orders, when I call GET /orders/me, then I receive an empty list with a 200, not a 404.

Ticket: FSE-110 **Priority:** Medium **Story points:** 3 **Phase:** Phase 1

Epic 6 — Concurrency safety with Redis (Phase 2)

US-11 — Atomic inventory decrement

*As a **system**, I want **to use a Redis atomic counter as the first gate for stock availability**, so that **inventory checks do not rely on database row locks under heavy concurrent load**.*

Acceptance criteria

- Given a sale starts, when the sale activates, then remaining_stock for each sale_inventory record is loaded into a Redis key.
- Given an order request arrives, when the system runs an atomic DECR on the Redis key, then a result below zero is immediately rejected as sold out without touching the database.
- Given the atomic decrement succeeds, when the order proceeds, then the order is recorded with status PENDING for asynchronous confirmation.

Ticket: FSE-201 **Priority:** High **Story points:** 8 **Phase:** Phase 2

US-12 — Enforce per-user purchase limit under concurrency

*As a **system**, I want **to track how many units a user has reserved for a sale_inventory in Redis**, so that **a user cannot bypass max_per_user by firing parallel requests**.*

Acceptance criteria

- Given a user has already reserved max_per_user units for a sale_inventory, when they send another order request, then it is rejected with a 409 before any stock decrement happens.
- Given two parallel requests from the same user each request the last allowed unit, when both arrive simultaneously, then only one is accepted.

Ticket: FSE-202 **Priority:** High **Story points:** 5 **Phase:** Phase 2

Epic 7 — Asynchronous order processing with Kafka (Phase 3)

US-13 — Queue confirmed reservations for processing

*As a **system**, I want **to publish a reservation event to a Kafka topic once Redis confirms stock availability**, so that **the API can respond instantly without waiting on the database write**.*

Acceptance criteria

- Given a successful Redis decrement, when the order request completes, then an order event (user, sale_inventory, quantity, flash_price) is published to the orders topic and the API responds 202 Accepted with status PENDING.
- Given the Kafka broker is temporarily unavailable, when publishing fails, then the system rolls back the Redis decrement so the unit is not lost.

Ticket: FSE-301 **Priority:** High **Story points:** 8 **Phase:** Phase 3

US-14 — Consumer finalises the order

*As a **system**, I want **a background consumer to read order events and write the final order record to MySQL**, so that **the database becomes the durable source of truth without blocking the customer-facing API**.*

Acceptance criteria

- Given an order event on the topic, when the consumer processes it, then an order row is written with status CONFIRMED and purchased_at set.

- Given the consumer fails to write to the database after several retries, when the retry budget is exhausted, then the event is moved to a dead-letter topic and the Redis-reserved unit is flagged for manual reconciliation.

Ticket: FSE-302 **Priority:** High **Story points:** 8 **Phase:** Phase 3

Epic 8 — Operations and quality (Phase 4)

US-15 — One-command local environment

*As a **developer**, I want **a docker-compose setup that brings up the API, MySQL, Redis, and Kafka together**, so that **the whole stack can be demoed or tested with a single command**.*

Acceptance criteria

- Given docker-compose up is run, when all containers report healthy, then the API is reachable on its configured port and connects to MySQL, Redis, and Kafka without manual setup.
- Given the containers are stopped and restarted, when MySQL volume is preserved, then existing data is retained.

Ticket: FSE-401 **Priority:** Medium **Story points:** 5 **Phase:** Phase 4

US-16 — Load test report proving zero overselling

*As a **developer**, I want **to run a load test simulating thousands of concurrent buyers against a small stock pool**, so that **I have evidence the system never oversells, suitable for a portfolio writeup**.*

Acceptance criteria

- Given a sale_inventory with total_stock = 100, when a JMeter or Gatling test fires 10,000 concurrent order requests, then exactly 100 orders end CONFIRMED and the rest receive 409 Sold Out.
- Given the test completes, when results are exported, then a report (response times, success/failure counts, final stock) is saved into the project README.

Ticket: FSE-402 **Priority:** Medium **Story points:** 5 **Phase:** Phase 4

3. Phased delivery plan

Phases are designed so each one introduces exactly one new piece of technology, motivated by a limitation discovered in the previous phase.

Phase	Sprint(s)	Focus & epics covered	New tech introduced	Exit criteria
Phase 1	Sprint 1–2	Core CRUD + synchronous order flow (Epics 1–5)	Spring Boot, JPA, MySQL	All APIs work via Postman; basic order flow prevents overselling at small scale using DB transactions
Phase 2	Sprint 3	Concurrency safety under load (Epic 6)	Redis (atomic DECR, per-user limit keys)	Load test with 1,000+ concurrent requests shows zero overselling
Phase 3	Sprint 4	Async order processing (Epic 7)	Apache Kafka	Order API responds in under 100ms; consumer reconciles all events to MySQL
Phase 4	Sprint 5	Hardening, packaging, and proof (Epic 8)	Docker Compose, JMeter/Gatling	One-command environment; published load test report in README

4. Sample sprint board — Sprint 1

Sprint 1 focuses on project setup, schema, and the read-side APIs that every later story depends on. Tasks (non-story tickets) are included alongside the user stories from Section 2.

Ticket	Type	Summary	Priority	Points	Story ref
FSE-001	Task	Initialise Spring Boot project: package structure, profiles, application.yml	High	1	—
FSE-002	Task	Design DB schema and write Flyway migration scripts (users, products, sales, sale_inventory, orders)	High	3	—
FSE-101	Story	User registration API	High	5	US-01
FSE-102	Story	View and update profile API	Medium	3	US-02

Ticket	Type	Summary	Priority	Points	Story ref
FSE-103	Story	Product catalog CRUD APIs	Medium	5	US-03
FSE-104	Story	Create and schedule a sale API	High	5	US-04
FSE-105	Story	Configure sale inventory API	High	5	US-05
FSE-008	Task	Global exception handler + request validation framework	Medium	2	—
FSE-009	Task	Postman collection + seed data script for local testing	Low	2	—

4.1 Sprint 2 — order flow and discovery

Ticket	Type	Summary	Priority	Points	Story ref
FSE-106	Story	Browse active and upcoming sales API	Medium	3	US-06
FSE-107	Story	View sale inventory with live stock API	Medium	3	US-07
FSE-108	Story	Place order API with transactional stock decrement	High	8	US-08
FSE-109	Story	Pessimistic-lock based overselling prevention + integration test	Critical	8	US-09
FSE-110	Story	Order history API	Medium	3	US-10
FSE-010	Task	Basic concurrent-request test script (small scale) to surface race conditions	Medium	3	—

5. Definition of done

A ticket is not "done" until all of the following are true:

- Code is committed with a message referencing the ticket ID (e.g. "FSE-108: add order placement endpoint").
- Unit tests cover the service-layer logic, including edge cases listed in acceptance criteria.
- The endpoint is exercised in Postman for both success and error responses, and the collection is updated.
- Database changes are captured as a Flyway migration, not manual SQL.
- No secrets or credentials are hardcoded — configuration comes from application.yml profiles or environment variables.
- README is updated if a new endpoint, environment variable, or setup step was introduced.

6. Non-functional requirements

Requirement	Target	Notes
Latency	Sale browsing and order placement APIs respond under 200ms at p95	Baseline measured in Phase 1; revisit after Redis/Kafka
Correctness	remaining_stock must never go negative; total CONFIRMED orders never exceed total_stock	Core invariant — validated by the Phase 4 load test
Concurrency	System handles at least 1,000 simultaneous order requests for a single sale_inventory without overselling	Validated in Phase 2 and Phase 4
Idempotency	Resubmitting the same order request (e.g. due to client retry) must not double-decrement stock	Introduce an idempotency key on the order endpoint by Phase 3
Observability	Health check endpoint and structured logs for order lifecycle (PENDING → CONFIRMED / FAILED)	Phase 4
Security	Order and profile endpoints require authentication; admin endpoints (product/sale management) require an admin role	JWT-based, can reuse approach from existing Redsky work

7. Out of scope (v1)

- Payment gateway integration — orders are simulated as confirmed without real payment capture.
- Frontend UI — all interaction is via REST APIs documented in Postman.
- Multi-warehouse or region-based inventory splitting.
- Refunds, returns, and order cancellation workflows.